AI Agent Guide

Smart Voicemail Router

Use this guide when an AI agent helps with the Smart Voicemail Router (SVR) repository. The project is a local-first competition prototype, not a production telecom system. The PRD, architecture, and team allocation are version 1.3. The dataset work is finished and the model selection is settled; the remaining open decisions are listed in the README. Unresolved decisions are not permission to guess.

1. Work in bounded tasks

Before starting, state the task's objective, approved inputs and versions, files in scope, expected output, and checks that will show it is done. Make one reviewable contribution at a time. Do not expand scope, change shared contracts, access new data, or make policy decisions without asking the human owner.

If a label, permission, retention rule, provider choice, schema behavior, or product policy is unclear, stop and identify the decision needed. Use synthetic fixtures for routine development when real data is not required.

2. Protect data and privacy

- Use only data whose source, license, consent, and competition rules allow the specific work. Confirm the approved location and dataset snapshot before accessing or copying recordings.
- Keep raw audio within the approved local workspace by default. Any external audio processing requires a separate explicit project decision after checking competition rules, data rights and consent, provider terms, and the approved scope. Masking a transcript does not anonymize its recording.
- Do not put recordings, identifying details, transcripts containing personal data, or secrets in prompts, code, logs, commits, or public artifacts. Keep credentials in approved environment variables or secret storage.
- External text processing is not an ordinary default. It requires a valid, durable approval recorded before transfer and matched to the exact provider, purpose, and payload. Minimize and locally mask the text first. Text approval never permits audio transfer.
- If the data permission or privacy boundary is uncertain, do not access or transmit the data. Ask the human owner.

3. EDA is done; do not change labels or splits

The shared EDA is done, the taxonomy is agreed, and the labels exist and are being audited by the team. The clean training set holds 5,751 rows: train 4,031, validation 1,136, test 584, seed 42. Do not change the label classes, annotation rules, evaluation split, or thresholds without Hazem. The audit page is the

mechanism for correcting labels: the team reviews them there every day, and the corrections are applied to the dataset in Phase 4.

When you work with the data, use the same authorized dataset snapshot. Check data rights and origin, file quality and language mix, missingness and duplicates, existing labels and ambiguity, privacy risks, and potential speaker/source leakage. Record the snapshot reference or checksum when permitted, commands or notebook version, assumptions, and limitations. Do not invent classes or silently freeze provisional labels. If data is unclear, missing, or unsupported, leave the value unknown and request human review.

4. Keep the decision fields separate

These are distinct fields, not synonyms:

- `complaint_category` : what kind of issue is reported.
- `complaint_urgency` : how time-sensitive or impact-heavy the issue is.
- `customer_state_text` : the customer's interaction or emotional state inferred from text, under approved definitions.

Keep `customer_state_text` , optional `voice_state_candidate` , and optional `customer_state_final` distinct. `customer_state_final` is a fused result and requires an approved policy. Voice cues are uncertain evidence, not ground truth, a diagnosis, or a substitute for complaint urgency. Queue priority is a separate system decision. Use only team-approved labels and signals; flag uncertainty or unsupported output for human review.

5. Make claims only from evidence

Never invent label definitions, dataset facts, permissions, model capabilities, metrics, test results, or production/security claims. Clearly mark unknowns and limitations. Report actual measurements with the dataset and split, sample counts, model/provider and configuration, runtime environment, metric, and relevant failures. Do not present targets or hypotheses as achieved results.

For each code change, provide the tests or other checks actually run, their commands and observed outcomes, and any untested behavior or limitations. A human team member must review the diff and evidence before accepting agent-generated work. Passing tests alone does not prove that requirements are met.

6. Run controlled experiments, with complete records

For an experiment intended to measure a change, vary **one experimental factor at a time**. Hold the dataset snapshot and split, prompt, schema, other configuration, and evaluation procedure constant as far as possible. Do not attribute a result to one factor if multiple factors or the data split changed. Study interactions only as separately identified experiments.

Record every run with:

- Run ID, hypothesis, and what outcome would support it.
- The single changed factor, including exact old and new values or model versions.
- Held-constant conditions: dataset version/checksum and split, prompt/schema/config versions, seed where applicable, and machine/runtime/provider.
- Command, code commit or branch, timestamps, artifact locations, and relevant approval ID for any permitted external text transfer.
- Sample counts, per-field metrics, per-stage latency, failures, qualitative review, and limitations.

This one-factor rule governs experimental ablations, not every code edit. A normal implementation change may touch multiple lines or variables. Keep it cohesive, testable, and reviewable, and describe its scope honestly.

Source of authority

This guide follows SVR-PRD-01, SVR-ARCH-02, and SVR-TEAM-03, all version 1.3. When a draft leaves a decision open or the documents conflict, do not guess: surface the question to the human project owner and wait for an approved decision.